

NEXUS Forge

곽현, Frontend Engineer

제조 이상 진단과 점검 결과 처리를 구현한 실시간 프론트엔드 공개 데모

배터리 코팅 라인의 이상을 찾고, 센서 신호를 비교해 작업 지시를 발행한 뒤 점검 결과와 이상 종결 사유를 기록합니다. React와 TypeScript로 실시간 시각화, 장애 복구와 업무 상태 전이를 구현했습니다.

공개 데모: 결정론적 합성 데이터를 사용합니다. 실제 설비 제어, AI 추론 결과나 고객사 운영 실적을 제공하지 않습니다.

[공개 데모](#) | [GitHub](#) | [기술 판단 사례](#) | [성능 측정](#)

검증: 현재 소스는 로컬에서 웹 108개, 서버 17개와 Chromium E2E 58개를 통과했습니다. 공개 데모는 main의 CI를 통과한 뒤 배포하며 /api/health에서 배포 SHA를 확인합니다.

먼저 볼 네 가지 구현

문제	해결 방식	근거
센서 피크가 사라지는 시계열 요약	구간 경계와 센서별 최솟값·최댓값 시점을 합치고 ECharts 추가 샘플링 비활성화	90N 피크 회귀
소켓 연결 중 센서 데이터만 멈춤	신선도와 하트비트를 따로 감시하고 근거가 유효하지 않으면 작업 지시 발행 보류	장애 복구 E2E
설비마다 다시 만드는 진단 화면	두 설비의 패널·기준값·안전 조건을 공통 구현에 연결하고 데이터 혼입 차단	두 설비 재사용
화면 이동과 다른 탭 저장에 따른 입력 유실	IndexedDB 트랜잭션, 필드별 충돌 검사, 탭 전용 초안과 명시적 취소 분리	다중 탭 회귀

핵심 업무 흐름

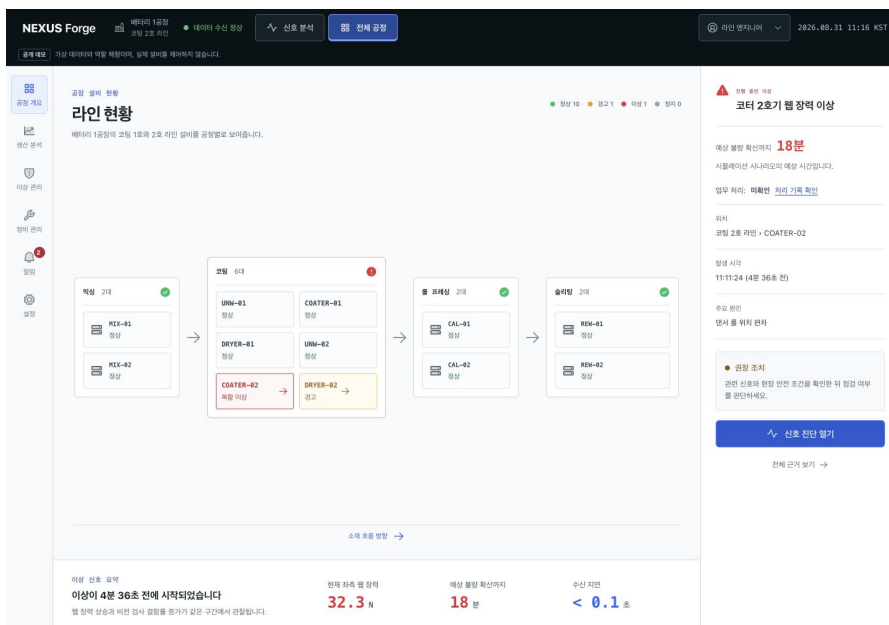
- 공정 개요에서 COATER-02 이상을 선택하고 동기화된 센서와 이상 발생 시점 참고값을 확인합니다.
- 안전 조건을 확인한 뒤 작업 지시를 발행하고, 정비 관리에서 점검 결과를 기록해 완료합니다.
- 연결 작업이 모두 끝난 뒤 잔여 위험과 사유를 남겨 이상을 별도로 종결합니다.
- DRYER-02로 전환해도 데이터와 작업 지시 발행 결과가 섞이지 않으며 새로고침 뒤 업무 이력을 복원합니다.

핵심 화면

실제 실행 화면의 센서값과 생산 실적은 합성 데이터이며, 작업 기록은 데모에서 입력한 내용입니다.

공정 개요 - 이상 설비에서 진단으로

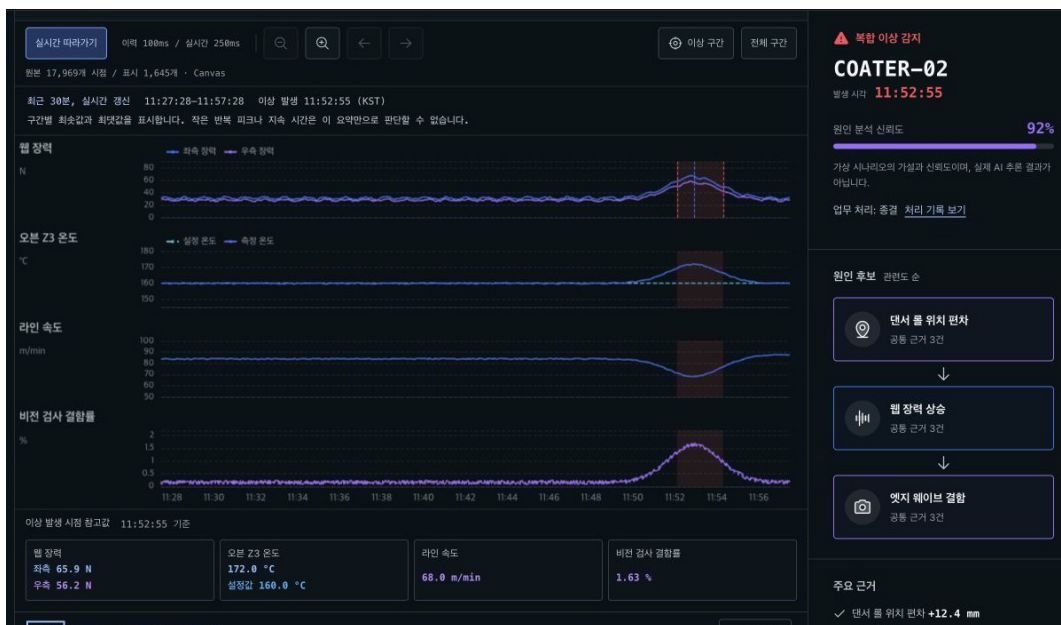
두 라인의 12대 설비를 공정 순서로 보여주고, 선택한 이상의 위치와 권장 조치를 진단 화면으로 연결합니다.



공정 개요: 12대 설비 상태와 코팅 2호 라인의 이상

신호 진단 - 같은 시간축에서 센서와 근거 비교

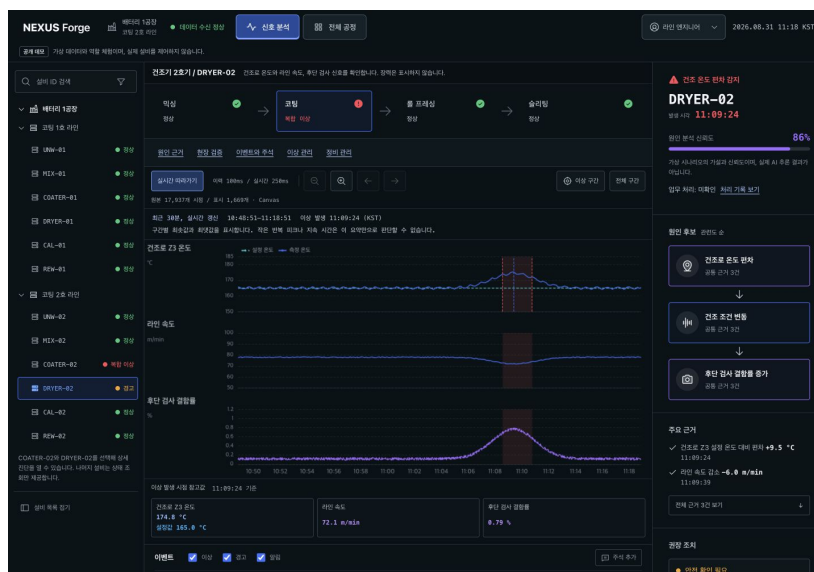
장력, 온도, 속도와 검사 결함률을 비교하고 이상 발생 시점 참고값, 이벤트와 합성 시나리오에서 설정한 원인 후보와 근거를 함께 표시합니다.



COATER-02 확대: 동기화된 네 개 차트, 이상 발생 시점 참고값과 원인 후보의 근거

두 번째 설비 재사용과 후속 처리

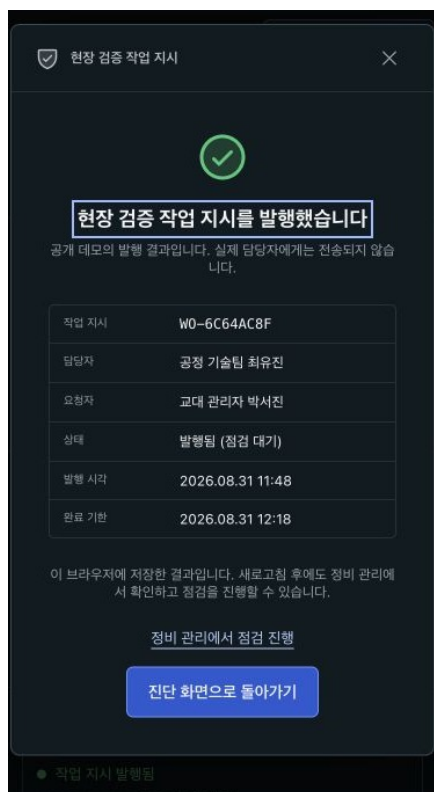
두 설비는 진단 페이지와 차트 렌더러, 작업 지시 발행과 상태 전이 로직을 공유합니다. DRYER-02는 장력 패널 없이 온도, 속도와 후단 검사 신호를 표시하고 165° C 설정값과 별도의 안전 조건을 적용합니다.



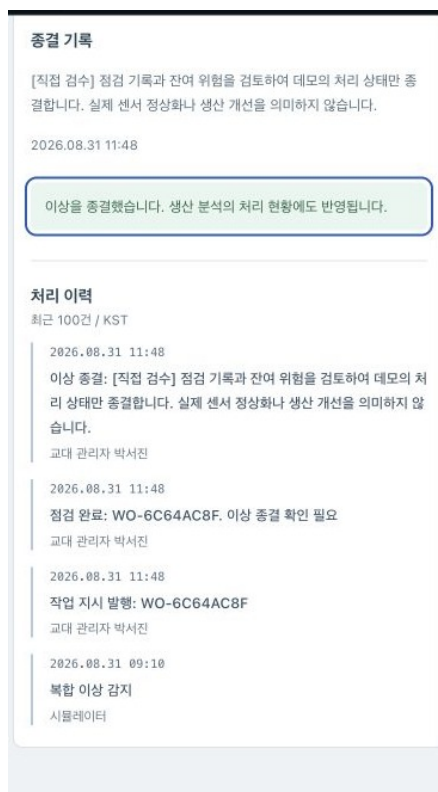
DRYER-02: 세 개 신호 패널과 설비별 원인 후보와 근거

작업 지시 발행 → 점검 완료 → 이상 종결

작업 번호, 담당자와 기한을 확인해 작업 지시를 발행한 뒤 점검 결과를 기록하고, 이상은 별도로 종결합니다. 작업을 완료해도 합성 수율이나 센서값은 바꾸지 않습니다.



작업 지시 발행 확대: 번호, 담당자와 기한 확인



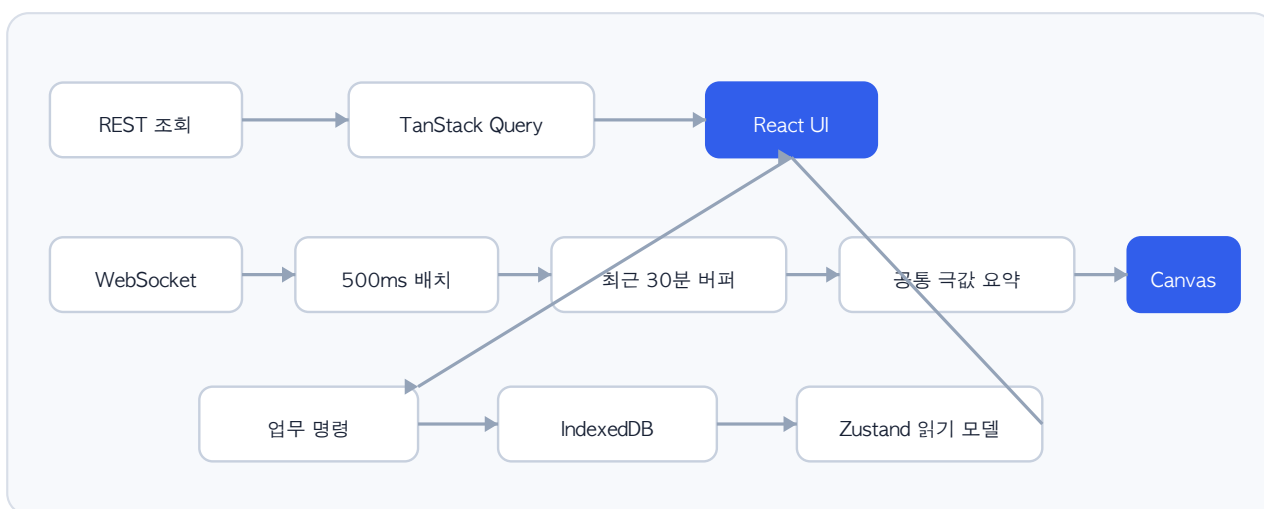
이상 종결 확대: 점검 완료와 별도의 처리 이력

두 설비의 처리 결과는 생산 분석에 미종결 이상 0건, 점검 완료와 이상 종결 각 2건으로 표시하며 합성 생산 실적과 구분합니다.

프론트엔드 구조와 시계열 처리

영역	구현
프론트엔드	React 19, TypeScript, Vite, TanStack Query, Zustand, ECharts Canvas
실시간 데이터	Node.js REST/WebSocket, 500ms 수신 배치, 최근 30분 링 버퍼
업무 상태	IndexedDB 트랜잭션, 탭 간 갱신, 입력 충돌 검사
품질	Turborepo, Vitest, Testing Library, Playwright, Storybook, GitHub Actions

TanStack Query는 조회·캐시를 관리하고 Zustand는 고빈도 센서 상태, 저빈도 업무 상태와 저장 전 입력을 분리합니다. IndexedDB 트랜잭션이 끝난 뒤에만 성공을 표시합니다.



시계열 요약의 보장 범위

공개 데모는 18,000개 시점을 받고, 성능 검증 API는 100,000개까지 허용합니다. 입력이 20,000개를 넘으면 전체 30분 구간을 최대 20,000개로 먼저 줄이고 차트에는 최대 1,800개의 공통 시점을 전달합니다. 구간 경계와 다섯 센서별 최솟값과 최댓값을 보존하지만, 모든 작은 피크의 반복 횟수와 지속 시간까지 보존하지는 않습니다.

10만 개 시점 차트 전후 비교

측정 항목	원본/전체 import	현재 요약/Core	변화
ECharts 번들, gzip	380.2KB	199.5KB	47.5% 감소
Canvas 렌더 중앙값	724.2ms	32.7ms	95.5% 감소
준비 + 렌더 중앙값	777.1ms	98.8ms	87.3% 감소
Canvas 렌더 p95	783.2ms	36.9ms	20회 표본

Apple M2, Chromium 151, 1200×700, DPR 1에서 CDP로 CPU 실행을 4배 느리게 제한했습니다. 2회 예열 후 20회 측정했으며 원본 100,000개와 선택된 1,732개 시점을 비교했습니다. 샘플링만의 중앙값은 62.1ms입니다. 물리 저사양 단말, 네트워크, React 갱신과 장시간 부하는 이 표에서 제외했습니다.

실제 앱의 진단과 작업 지시 발행 흐름

프로덕션 빌드와 REST/WebSocket 런타임에서 100,000개 시점 응답을 사용했습니다. 두 설비를 각각 3회, 30초씩 측정한 뒤 COATER-02를 60분 관찰했습니다. Apple M2, Chromium headless, 1440×1024, DPR 1, CPU 4배 제한과 로컬 HTTP/WS 조건입니다. 공개 Vercel이나 물리 저사양 단말 결과는 아닙니다.

측정 구간	통계 / 표본	COATER-02	DRYER-02
설비 선택 → 첫 이력	중앙값 / 3	1,201.5ms	1,286.2ms
이력 요청 → 파싱·검증	중앙값 / 3	614.7ms	405.9ms
이력 요청 → 첫 반영	중앙값 / 3	844.0ms	896.4ms
최신 수신 → 차트	p95 / 180	308.7ms	305.6ms
배치 상태 → 차트	p95 / 180	90.6ms	76.9ms
작업 지시 발행 → 결과	중앙값 / 3	179.5ms	122.9ms

차트 반영 시간은 ECharts의 `finished` 이후 두 번의 렌더 프레임 기회까지이며 실제 픽셀 합성 완료 시각은 아닙니다. 60분 실행에서 최신 수신→차트 반영 7,200개 표본의 p95는 303.8ms였고, 상태 반영→차트 반영 p95는 79.2ms였습니다. 718개 체크포인트에서 보존 시점은 7,179–18,106개, 표시 시점은 1,574–1,776개였습니다.

관찰 구간 Long Task는 29건, 최댓값은 276ms, 50ms 초과 누적 차단은 916ms였습니다. 강제 GC 뒤 사용 힙은 22.3MB에서 28.9MB로 6.6MB 증가했습니다. 오류와 실패 요청은 0건이지만 제한된 1시간 관찰은 메모리 누수 부재나 8시간 교대 안정성을 증명하지 않습니다.

기여 범위

2026년 8월 30일 시작한 개인 프로젝트입니다. 지원 포지션과 개발 범위, 개선 우선순위, 공개 데모 표시 기준을 정했습니다. 실제 화면에서 발견한 결함의 재현 조건과 완료 기준을 기록하고, 수정 후 테스트와 배포 결과를 확인했습니다. GitHub 저장소 생성과 Vercel 연결도 직접 진행했습니다.

제품 방향과 디자인 대안을 탐색하고 구현, 테스트, 문서를 작성하는 과정에서 AI 개발 도구를 활용했습니다. 작성자는 요구사항 충족 여부, 검증 결과와 공개 범위를 최종 확인했습니다. 이 프로젝트를 실제 제조 고객사 운영 경험이나 모든 코드를 수작업으로 작성한 사례로 제시하지 않습니다. [기술 판단 사례](#)에 선택한 대안과 검증 비용을 공개합니다.

검증과 공개 범위

검증	결과와 범위
현재 소스 로컬	웹 108개, 서버 17개, lint와 TypeScript 검사, 전체 빌드 통과
업무 흐름	현재 소스의 Chromium E2E 58개 통과. 실패·생략·재시도 0개
화면 검수	일곱 화면과 두 설비의 작업 지시 발행 → 점검 완료 → 이상 종결 → 새로고침 복원 확인. 모바일 16조건과 키보드 조작 확인
빌드와 배포	전체 빌드, Storybook과 Sites 호환성 검사 통과. Vercel 자동 배포와 SHA 일치 확인

공개 배포

main의 GitHub Actions 검증 성공 후에만 배포합니다. E2E가 재시도 후에야 통과한 경우에도 배포를 차단합니다. 배포 뒤에는 /api/health SHA와 커밋을 대조하고, 8개 화면 경로와 두 설비의 REST 이력 조회 및 WebSocket 수신을 확인합니다.

운영 한계

- 실제 PLC, MES, SCADA, OPC-UA, MQTT 연결이나 AI 추론 엔진이 없습니다. 합성 시나리오는 실제 라인의 물리적 인과 모델이 아닙니다.
- 업무 기록은 브라우저 IndexedDB에만 저장하며 기기 간 동기화, 서버 백업, 인증·권한·승인·불변 감사 로그를 지원하지 않습니다.
- 서버가 반환한 작업 지시 발행 결과는 인스턴스 메모리에 최대 100건만 남습니다. 물리 저사양 단말, 실제 제조 네트워크, 다중 사용자와 8시간 교대 규모는 검증하지 않았습니다.

검증 근거와 재현

근거	확인 범위
차트 스트레스	10만 개 시점, CPU 4배 제한, 20회 표본의 Canvas 렌더와 번들 비교
실제 앱 스트레스	두 설비의 3회 흐름과 COATER-02 60분 수신, 합·Long Task·오류·소스 해시
업무 회귀	두 설비의 진단, 작업 지시 발행, 복구, 모바일과 접근성 조건을 포함한 E2E 58개

[차트 원본](#) | [60분 앱 원본](#) | [성능 측정 방법](#)

설계 배경과 다음 검증

[지신 공식 사이트](#)와 공개 R&D 정보, 채용 공고를 참고해 독립적인 제조 운영 데모를 설계했습니다. Apex OS 비공개 화면이나 고객사 내부 정보를 사용하지 않았습니다. 실제 커넥터, Ontology 탐색, 인증과 2D/3D 공장 시각화는 미구현 확장 범위입니다.

[공개 데모](#) | [GitHub 저장소](#) | [최종 검증 기록](#) | [두 설비 재사용](#)

곽현, Frontend Engineer

[GitHub: kwakhyun](#)